

PHOEBUS: AN OPEN ECOSYSTEM FOR CONTROL SYSTEM APPLICATIONS AND SERVICES

K. Shroff, Brookhaven National Laboratory, Upton, NY, USA

R. Lange, ITER Organization, St. Paul lez Durance, France

T. Ford, Lawrence Berkeley National Laboratory, Berkeley, CA, USA

G. Weiss, S. Brewer, European Spallation Source ERIC, Lund, Sweden

K. Kasemir, Oak Ridge National Laboratory, Oak Ridge, TN, USA,

I. Finch, Science and Technology Facilities Council, UK

Abstract

The Phoebus toolkit continues to evolve as a modern, extensible platform for control system user applications and middle-layer services. As the next generation of the Control System Studio, Phoebus retains the familiar, integrated toolset experience while replacing the Eclipse RCP framework with a modular architecture built on standard Java technologies and JavaFX. This transition simplifies maintenance and extensibility while providing foundational building blocks for scalable services and modern applications.

Recent work has focused on strengthening infrastructure and streamlining deployment. These efforts include support for recent Java LTS releases, modernization of microservices with updated Kafka and Spring Boot, and expanded documentation. Core services—covering alarm management, save/restore, name resolution, and logbooks—continue to mature, with emphasis on easier configuration, improved scalability, and alignment with modern web standards and containerized workflows. The Phoebus collaboration now

includes contributors from dozens of facilities worldwide, many of them first-time participants. Alongside technical progress, the project prioritizes a sustainable and inclusive collaboration model to support future developers and users.

INTRODUCTION AND MOTIVATION

Phoebus is the successor of Eclipse as the framework for Control System Studio and a growing number of control system services [1, 2]. Its development was motivated by the need to simplify the architecture, remove the heavy dependency on the Eclipse RCP framework, and modernize the technology stack. The result is an ecosystem that is both extensible and sustainable, as seen in Fig. 1, and which has been adopted by an increasing number of facilities worldwide.

At the heart of Phoebus is the integrated user experience. The toolkit provides a cohesive set of tools and applications that are easy to navigate between, where data can be seamlessly transferred from one tool to another, ensuring a consistent experience that

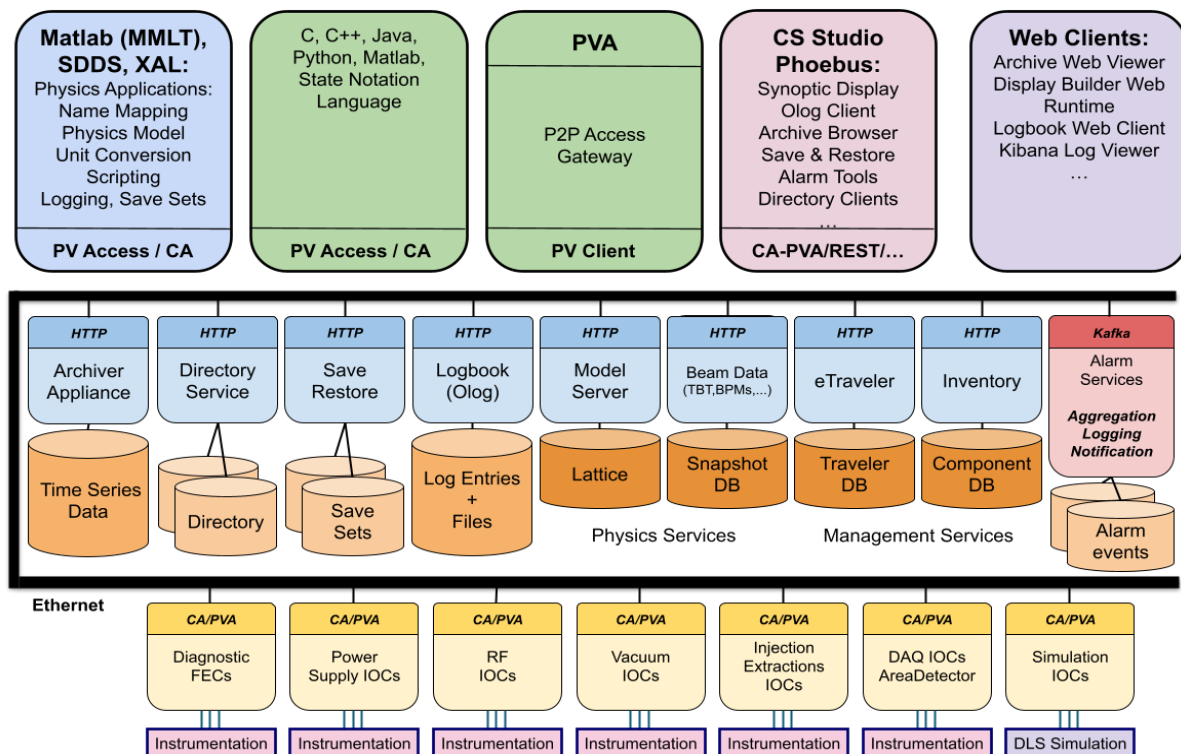


Figure 1: EPICS controls ecosystem showing Phoebus alongside IOCs, services, and clients.

supports the user workflows within the same environment. For example, an operator can investigate the details of an alarm, retrieve historical data for the PVs that triggered it, open related OPI/HMI displays to interact with devices, and finally record the investigation in the logbook. Each of these steps uses different applications—Alarm UI, Data Browser, Display Builder, and Logbook—but Phoebus enables seamless transitions between them. Selection services and adapter services automatically transfer context including PV names, current values, timestamps, alarm states, archive sources for historic data, OPI screen locations, log entries, and more. This rich context propagation reduces user effort and improves efficiency.

For developers, Phoebus provides a framework for creating applications that integrate smoothly into this ecosystem without requiring tight dependencies on other applications. Shared modules offer access to optimized resources such as connections to EPICS Process Variables (PVs), REST clients, and other common services, reducing duplication and lowering the barrier to entry for new contributors.

Community adoption has also been a central motivation. The project emphasizes open collaboration and inclusivity, encouraging contributions from new developers while maintaining a consistent architecture. This ensures that Phoebus continues to evolve in response to user needs while remaining sustainable in the long term.

To understand the motivation for this architecture, it is helpful to situate Phoebus within the broader controls ecosystem used at accelerator and large-scale scientific facilities. As illustrated in Figure 1, instrumentation and Input/Output Controllers (IOCs) at the lowest level communicate via EPICS protocols such as Channel Access and PVAccess [3]. Middle-layer services—including the archiver, logbook, save-and-restore, and alarm system—provide persistence, workflow integration, and operational infrastructure. On top of these, multiple client environments coexist: physics toolkits, scripting interfaces, web clients, legacy CS-Studio, and increasingly, Phoebus. This context highlights the dual role of Phoebus: providing operator-facing applications and frameworks for new tools, while also offering middle-layer services that integrate tightly with existing site infrastructure.

ARCHITECTURE OF THE PHOEBUS FRAMEWORK

Phoebus is organized as a modular framework with a small set of reusable core modules and a flexible extension mechanism. The design balances stability and adaptability: stable libraries and APIs provide common behavior, while the extension model allows sites to tailor distributions with site-specific applications, protocols, and service adapters. Figure 2 shows the overall architecture of the framework.

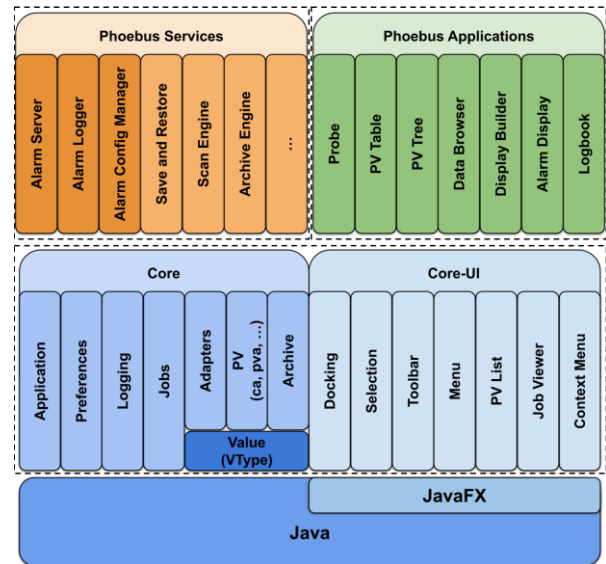


Figure 2: Phoebus Architecture. Core and Core-UI libraries provide the foundation for both Phoebus applications and Phoebus services.

Overview

At the foundation, Phoebus builds on standard Java and JavaFX technologies rather than Eclipse RCP. Above this runtime, the framework is divided into three tiers:

- Core modules – protocol-agnostic data access, value models, adapters, job scheduling, logging, and configuration.
- Core-UI modules – reusable UI libraries such as docking layouts, selection services, menus, and toolbars.
- Applications – operator-facing tools (e.g., Display Builder, Data Browser, Alarm UI, Logbook UI) that build on both the core and core-UI libraries.
- Middle-layer services - such as the Alarm Server, Save & Restore, and Archive Engine. These services are built on the same core modules but exclude UI dependencies.
- Product – an assembled distribution of Phoebus core modules and applications, tailored to form a desktop toolkit for users.

Extension mechanism (SPI and plugin discovery)

Extensibility in Phoebus is based on Java’s Service Provider Interface (SPI). This mechanism makes it possible to add new capabilities seamlessly while reusing the functionality already provided by the core modules. Developers or site integrators can contribute applications, data-sources for new protocols, clients to middle layer services, or other extensions simply by implementing the relevant SPI contract. The runtime then discovers and incorporates these contributions automatically.

This flexible design enables tailored deployments for different facilities. For instance, one site may implement a custom logbook service, while another provides a specialized data-source adapter or a new operator tool.

Because integration relies on well-defined contracts, extensions stay loosely coupled and can be added or removed as needed, preserving the core's stability and enabling curated site-specific products. Here is a (non-exhaustive) list of what can be achieved through SPI hooks:

- Core level – contribute data sources to new protocols, or add clients for different middle-layer microservices.
- Application level – introduce new applications, register file types that applications can handle, and define how applications expose their context to others.
- UI level – extend toolbars, menus, and context menus with custom actions.

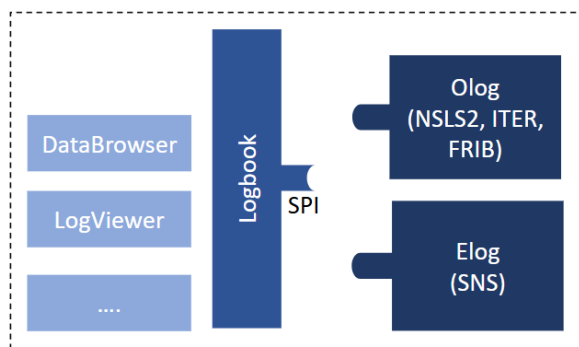


Figure 3: Example SPI for Logbook integration.

All of these integration points demonstrate how SPI enables new functionality to be woven into Phoebus. Figure 3 illustrates this with the Logbook SPI: applications interact with a generic logbook API, while SPI providers map these calls to concrete client calls for services like Olog or elog. This example highlights how the same application code can remain service-agnostic allowing sites to select the service implementation that fits their environment.

Core modules

Core modules provide the supporting infrastructure for business logic across applications and services. These include:

- CorePV / Data-source layer – centralizes all protocol interactions. It provides a consistent API for reading and writing values, subscribing to updates, and accessing PV metadata. Underlying protocol adapters (CA, PVA, simulators, vendor systems) are discovered via SPI. CorePV reuses connections across subscribers and encapsulates reconnection logic, reducing network load and simplifying application development.
- VTypes / Value model – define canonical value representations. Each VType is immutable and carries both the raw value and its context—timestamp, alarm state, engineering units, and display ranges. VTypes cover scalars, arrays, enumerations, strings, and tables. They can be

serialized for REST or WebSocket transport, enabling interoperability between applications and services.

- Job scheduler – provides controlled background execution, ensuring tasks run off the UI thread for responsiveness.
- Formula pipelines – allow values to be processed and transformed through configurable pipelines, supporting calculations and derived signals.
- Selection and Adapter framework – enables rich cross-application workflows. For example, selecting an alarm in the Alarm UI can directly open a probe on the affected PV, or launch a Data Browser view with historical trends. This is achieved by publishing the current selection and using adapter factories to map to other domain types. Applications remain loosely coupled while still interoperating.
- Preferences and configuration – layered product, site, and user preferences for consistent behavior across deployments.
- Logging and diagnostics – structured logging utilities, health checks, and developer diagnostics.
- Security module – provides services for managing credentials and secure access to resources. This allows applications and services to handle sensitive information such as passwords and tokens in a standardized, centralized way.
- Utility module – a collection of general-purpose helper classes and functions used across the framework, reducing duplication and supporting consistency.

Core-UI modules focus on common user interface behaviors and integration points. They include:

- Docking and UI primitives – consistent docking layouts, persistent workspaces, and shared widgets.
- Menu, toolbar, and context integration – SPIs for contributing UI elements such as menu actions and toolbar entries.
- Selection services for UI – managing current selections and context menus in user-facing applications.
- Logbook SPI – provides backend-agnostic operator logging. Applications such as Alarm UI, Save & Restore, and Display Builder call the logbook API, which delegates to whichever provider is active. Log entries automatically capture context, ensuring rich records while application code remains backend-independent.

These modules exemplify the philosophy of separating stable abstractions from site-specific implementations and ensure that application developers can focus on business logic while leveraging shared infrastructure.

Phoebus Product

Phoebus distributions are delivered as curated “products”: collections of JARs assembled with a product descriptor. Each site can deliver a tailored distribution

containing only the relevant applications, protocols, and service adapters. Because SPIs handle discovery, adding or removing functionality is as simple as changing the product composition. Products can also be branded with site-specific icons, default preferences, and menus. For containerized or kiosk-style deployments, only the required components are included, reducing complexity and attack surface.

PHOEBUS APPLICATIONS

Phoebus provides a comprehensive suite of operator-facing applications that are tightly integrated with EPICS middle-layer services. The emphasis is on a consistent user experience across displays, trending, alarms, and logging, while maintaining compatibility with existing infrastructure. Applications interoperate by sharing context such as PV names, values, alarms, and archive sources, enabling operators to move fluidly between tools without duplication of effort.

The Display Builder is a cornerstone application, combining an interactive editor with a runtime for executing control system displays [4]. It offers a wide set of widgets for monitoring and controlling PVs, designed to work out-of-the-box with alarm awareness, units, and precision. Advanced customization is supported through properties and scripting, while the XML-based “*.bob*” format ensures portability. Displays from legacy tools such as EDM, MEDM, and BOY can be auto-converted with minimal adjustments, and the same files are usable in the Display Builder Web Runtime.

For time-series visualization, the Data Browser provides trending across multiple SPI-based archive providers, including the EPICS Archiver Appliance as well as RDB- and TimescaleDB-backed history stores. This allows operators to retrieve past data alongside live updates in a unified plotting environment.

Phoebus also includes several PV utilities: Probe for detailed PV introspection, PV Table for monitoring and save/restore of groups of PVs, and PV Tree for visualizing EPICS record linkages. These tools support diagnostics, system exploration, and day-to-day operations.

Alarm awareness is built into all applications, but the Alarm User Interfaces provide a dedicated view of system alarms [5]. Components include high-level overviews, tables of active alarms, hierarchical alarm trees, and an annunciator for voice notifications. Together, they ensure critical conditions are surfaced to operators in real time and managed efficiently.

Supporting discovery and configuration management, the Channel Finder clients offer fast, case-insensitive search of PV names enriched with metadata such as IOC host, record type, and status. Meanwhile, the Save-and-Restore application provides snapshot and recovery functionality: users can capture sets of PV values, later restore them, and even scale or merge snapshots to reproduce known states or reconfigure subsystems quickly.

By combining these applications into a unified environment, Phoebus delivers more than the sum of its

parts. Operators can move seamlessly from investigating an alarm, to browsing related history, to interacting with device displays, and finally documenting findings in a logbook—all within the same toolkit. This integration of applications with middle-layer services makes Phoebus a practical and sustainable environment for facility operations.

PHOEBUS SERVICES

Modern control systems must handle demanding and varied operational requirements, such as archiving large numbers of control system signals, managing alarms across distributed facilities, and providing online logbooks for operators and scientists. Implementing these capabilities as a set of modular microservices offers several advantages:

- Loose coupling and modularity: Each service addresses a well-defined concern, making it possible to evolve or replace individual components without disrupting the overall system.
- Independent scalability: Services that experience higher load, such as archiving or alarm handling, can be scaled independently of others.
- Resilience and fault isolation: Failures are contained within the affected service, reducing the risk of cascading outages across the control system.
- Alignment with modern CI/CD practices: Microservices can be built, tested, and deployed independently, enabling more frequent and reliable software updates.
- Technology flexibility: Each service can be tailored to its use case by selecting the most appropriate data store or infrastructure. For example, an archiver may rely on time-series databases optimized for high-rate numerical data, while a logbook service may use relational or NoSQL document databases suited to natural language entries and attachments. This flexibility extends to facilities themselves, allowing each site to architect a controls technology stack that best addresses its specific needs and operational environment.

These properties make the microservices architecture ideally suited for developing a mesh of services that extend and augment the EPICS control system in a sustainable, scalable, and maintainable way.

Architecture

While the Phoebus ecosystem encompasses a technically diverse set of middle-layer services, the design philosophy emphasizes standardization where possible and flexibility where necessary.

Many services are implemented using the Spring Boot framework, which has become a de facto standard within the ecosystem. Spring Boot simplifies the process of building and maintaining services by providing out-of-the-box support for security, persistence layers, database access, embedded HTTP servers, and production-ready monitoring and configuration tools. By relying on this

foundation, developers can focus on service-specific functionality rather than re-implementing common infrastructure. This commonality also lowers the barrier for contributors, reduces maintenance overhead, and makes operational support more predictable.

At the same time, not all services follow this model. Certain high-throughput or low-latency use cases require specialized approaches—for example, services that adopt a publish–subscribe model using message buses, or clusterable services designed for scaling time-series data storage. This technical diversity ensures that each service can employ the most suitable technology for its role, while interoperability is preserved through well-defined APIs and shared data models.

The Phoebus core libraries further strengthen this consistency. In addition to providing common data types (e.g., VTypes and CorePV), these libraries encapsulate reusable logic such as data serialization, connection handling, and process variable access. This not only ensures that services and clients exchange data in predictable formats but also accelerates development by freeing service authors from having to re-implement low-level functionality.

CI/CD practices are taken into consideration across the ecosystem, with automated builds, testing, and packaging forming the foundation for reliable releases. While implementation details vary between sites, the ecosystem provides the building blocks for containerized deployment, enabling services to be integrated into modern orchestration frameworks such as Kubernetes. This approach simplifies adoption and provides a path toward scalable, cloud-native operations.

The result is an ecosystem that balances standardization for ease of development with diversity for performance and scalability requirements.

EXAMPLES

A variety of microservices have been developed within the Phoebus ecosystem, each addressing a distinct aspect of facility operations. Together they provide persistence, discovery, alarm management, and logging capabilities that extend the underlying EPICS control system.

Alarm Services

The alarm system is composed of several cooperating services that together provide comprehensive alarm management [5].

- **Alarm Service:** The central component responsible for monitoring a configurable set of process variables (PVs) and notifying operators of new, pending, and acknowledged alarms through the alarm user interface.
- **Alarm Logger:** Records a historical timeline of alarms and associated operator actions, such as acknowledgements. In addition to supporting post-mortem analysis, this data enables the generation of statistics that identify noisy alarms and uncover correlations across multiple alarms. Such insights are

essential for refining configurations and reducing operator burden.

- **Alarm Configuration Logger:** Maintains a versioned history of alarm configuration changes, allowing administrators to track modifications and, if necessary, restore earlier snapshots.

Together these services provide a resilient, data-driven alarm system that supports both immediate operational needs and long-term configuration optimization.

Save and Restore Service

The Save and Restore service provides the ability to capture snapshots of system state and later restore them. Snapshots and associated metadata are persisted in Elasticsearch, which also powers efficient search and filtering by attributes such as snapshot name, date, user identity, or user-defined tags. Access is provided via a REST API, enabling seamless integration with client applications. This functionality is critical for reproducibility in operations and experiments, as well as for rapid recovery from configuration changes or failures.

Phoebus Olog

Phoebus Olog is an online logbook service designed for persistent and searchable operator logs [6]. Log entries are stored in Elasticsearch, while attachments such as images and files are managed in MongoDB. Clients interact through a REST API that supports both metadata queries and natural language search across log contents.

The logbook ecosystem includes multiple clients to serve different user needs:

- an integrated client within the Phoebus framework, providing seamless context sharing with other applications;
- a web client implemented in HTML/JavaScript for browser-based access;
- dedicated smartphone clients optimized for mobile workflows.

This multi-platform approach ensures that operators and scientists can access and contribute to the logbook from anywhere in the facility.

Channel Finder

The Channel Finder service acts as a directory for process variables within the otherwise flat EPICS namespace [7]. It supports discovery through tags, properties, and a query-by-functionality approach, enabling users to locate channels based on criteria such as device type or functional grouping. Channel Finder also facilitates the construction of hierarchical views over the namespace, improving navigability for both high-level physics applications and data management tools. By providing a searchable, metadata-driven catalog of PVs, it enhances the accessibility and flexibility of large-scale control systems.

Archiver Services

Archiving of control system data is a fundamental requirement for analysis, diagnostics, and operations.

Within the EPICS and Phoebus ecosystem this need has been addressed through multiple implementations of the archiver service, each reflecting different design choices and deployment scales.

The Phoebus Archive Engine provides a straightforward solution that integrates closely with other Phoebus services. It is particularly attractive to facilities with in-house expertise in relational database management, as it can leverage backends such as PostgreSQL with the TimescaleDB extension for efficient partitioning and optimized time-series retrieval. This makes it well suited for moderate-scale deployments where simplicity and familiarity are priorities.

In contrast, the EPICS Archiver Appliance is designed to scale horizontally, supporting millions of process variables through clustering and distributed storage. It achieves high performance by storing data in protocol buffer-based binary files, enabling both compact storage and fast retrieval. Its architecture supports high-throughput ingest while providing advanced query capabilities, making it the preferred choice for facilities with large-scale archiving requirements.

By accommodating multiple implementations, the ecosystem allows each facility to adopt the solution that best matches its operational scale, infrastructure expertise, and performance needs.

COMMUNITY AND COLLABORATION

The Phoebus project continues to grow as a collaborative, community-driven effort. A steady increase in contributors, including many first-time contributors, reflects not only the openness of the project but also the fact that the infrastructure and codebase are significantly easier to understand and extend than the earlier Eclipse RCP-based framework. This lower barrier to entry has made it possible for a broader set of developers from different institutions to participate.

Community engagement is reinforced through workshops, training sessions, and collaborative development events that help new users and developers quickly become effective contributors.

ROADMAP

Ongoing development of Phoebus continues with a focus on modernizing and extending the technology stack—from user-facing applications and middle-layer services to the underlying infrastructure—ensuring the platform continues to evolve with the needs of the control system community. Current and planned work includes:

- Enhanced Display Builder widgets, expanding the expressiveness and usability of operator interfaces.
- EPICS 7 and PVAccess support, keeping pace with community developments such as enhanced security features, IPv6 support, and simplified name resolution.
- Infrastructure and deployment enhancements, including migration to newer Java LTS versions (17+), upgrades to Spring Boot and Kafka,

improvements across middle-layer microservices, and adoption of containerized distribution with CI/CD pipelines.

- Operational observability, through standardized logging, metrics, and Grafana dashboards for monitoring large-scale deployments.

We continue to support facilities that are still in the process of migrating, while also drawing on the experiences of those that have already transitioned—including ESS, FRIB, and SNS—to guide future improvements and ensure Phoebus remains a robust and adaptable platform for control system development.

CONCLUSION

Phoebus is the culmination of decades of experience and expertise derived from the CS-Studio collaboration. It is rooted in a deep understanding of the control system community's needs and the practical challenges of operating large-scale scientific facilities.

Through its modular architecture and microservice-oriented design, Phoebus has matured into a modern, extensible, and sustainable technology stack. It combines flexibility for developers with robustness for operations, making it well suited to meet current demands while remaining adaptable to future innovations. This strong foundation ensures that Phoebus will continue to evolve alongside advances in control system architectures and technology, providing a reliable platform for the next generation of tools and services.

ACKNOWLEDGEMENTS

The Phoebus project builds on the long history of collaboration established by CS-Studio and has benefited from the contributions of many individuals and institutions worldwide. While the authors represent only a subset of the active developers, we gratefully acknowledge the efforts of all contributors—past and present—whose work has made Phoebus a sustainable and evolving platform for the control system community.

REFERENCES

- [1] Phoebus, <http://www.phoebus.org>
- [2] K. Shroff et al., “Phoebus Tools and Services,” in Proc. 19th Int. Conf. Accel. Large Exp. Phys. Control Syst. (ICALEPCS'23), Cape Town, South Africa, Oct. 2023, pp. 944–948, paper TUSDSC08. doi: 10.18429/JACoW-ICALEPCS2023-TUSDSC08\
- [3] EPICS Base releases, <https://epics-controls.org/>
- [4] K. Kasemir, M. Grodowitz, CS-Studio Display Builder, ICALEPCS 2017
- [5] K. Kasemir, CS-Studio Alarm System based on Kafka, ICALEPCS 2019
- [6] K. Shroff, G. Weiss, Phoebus Olog, EPICS Spring Collaboration Meeting 2021
- [7] K. Shroff et al., ChannelFinder, EPICS Spring Collaboration Meeting 2023